Fourier Transform in Image Processing: Magnitude vs Phase Analysis

Full Documentation — Line-by-Line Code Explanation with Results

TA 2025–2026 • Signal & Image Processing

Table of Contents

1. Introduction — Why This Matters
2. Background Theory
3. Step 1: Load and Prepare the Images
4. Step 2: Compute the 2D Fourier Transform
5. Step 3: Extract Magnitude and Phase
6. Step 4: Reconstruct with Swapped Components
7. Step 5: Display Original Images
8. Step 6: Display Magnitude Spectra
9. Step 7: Display Phase Spectra
10. Step 8: The Key Result — Visual Proof
11. Step 9: Quantitative Verification
12. Conclusion & Key Takeaways
13. References

1. Introduction — Why This Matters

The Fourier Transform is one of the most fundamental tools in signal and image processing. It converts a signal from the **spatial domain** (pixel values) into the **frequency domain** (sinusoidal components), revealing information that is invisible in the original representation.

Every frequency component in a Fourier-transformed image has two parts: a **magnitude** and a **phase**. A natural question arises:

> **The Central Question:** If we had to choose only one — magnitude or phase — which component carries the information that allows us to *recognize* the image?

This experiment provides a definitive, visual answer. Understanding this result is critical because:

- **Image compression**: Knowing which component matters more informs how we allocate bits
- **Image reconstruction**: Phase retrieval algorithms are essential in MRI, X-ray crystallography, and astronomy
- **Feature extraction**: Understanding that phase = structure helps design better edge detectors and feature descriptors
- **Watermarking & steganography**: Phase-based methods are more robust than magnitude-based ones
- **Optical engineering**: Lens design, holography, and adaptive optics all rely on phase information

2. Background Theory

The **2D Discrete Fourier Transform (DFT)** of an image *f(x, y)* of size *M* × *N* is:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \cdot e^{-j2\pi(ux/M + vy/N)}$$

Each complex coefficient F(u,v) can be expressed in polar form:

$$F(u, v) = |F(u, v)| \cdot e^{j \cdot \varphi(u,v)}$$

Component	Symbol	Formula	What It Encodes
Magnitude	|F(u,v)|	√(Real² + Imag²)	Amplitude/energy of each frequency — controls contrast, brightness distribution

Phase	φ(u,v)	atan2(Imag, Real)	Position/alignment of each frequency — controls edges, shapes, textures, spatial structure

The **inverse DFT** reconstructs the image from its frequency components. The key insight is that you can *mix* the magnitude from one image with the phase from another, and the inverse DFT will still produce a valid image — letting us see which component "wins."

3. Step 1: Load and Prepare the Images

Code

```
38  clc;            % Clear the command window
39  clear all;      % Remove all variables from the workspace
40  close all;      % Close all open figure windows
```

> **Lines 38–40:** These three commands reset MATLAB to a clean state. `clc` clears the Command Window text. `clear all` removes every variable from memory so there are no leftover values from previous runs. `close all` closes any open figure windows.

```
48  imageA = imread('cameraman.tif');
51  imageB = imread('circuit.tif');
```

> **Lines 48 & 51:** `imread()` reads an image file from disk and stores it as a matrix of pixel values. Each pixel is stored as a `uint8` integer (0–255). `cameraman.tif` is a classic 256×256 grayscale test image. `circuit.tif` is a grayscale image of a printed circuit board. Both ship with MATLAB's Image Processing Toolbox.

```
56  imageA = double(imageA);
57  imageB = double(imageB);
```

> **Lines 56–57:** `double()` converts the pixel values from `uint8` (integers 0–255) to **double-precision floating point** numbers. This is essential because the FFT involves complex arithmetic (multiplication, exponentiation) that requires full numerical precision. Without this conversion, MATLAB would throw errors or produce incorrect results.

```
62  [rowsA, colsA] = size(imageA);
63  imageB = imageB(1:rowsA, 1:colsA);
```

> **Lines 62–63:** `size()` returns the dimensions (rows, columns) of Image A. We then **crop Image B** to match those dimensions. This is critical because when we swap magnitude and phase between two images, the matrices must be the same size — you can't combine a 256×256

magnitude with a 280×300 phase. We use `imageB(1:rowsA, 1:colsA)` to keep only the top-left portion matching Image A's size.

```
66  fprintf('=== Image Information ===\n');
67  fprintf('Image A (cameraman): %d x %d pixels\n', rowsA, colsA);
68  [rowsB, colsB] = size(imageB);
69  fprintf('Image B (circuit):   %d x %d pixels (after cropping)\n', rowsB, colsB);
```

Lines 66–69: `fprintf()` prints formatted text to the Command Window. `%d` is a placeholder for integers. This confirms both images are the same size (256×256) after cropping, serving as a quick sanity check before proceeding.

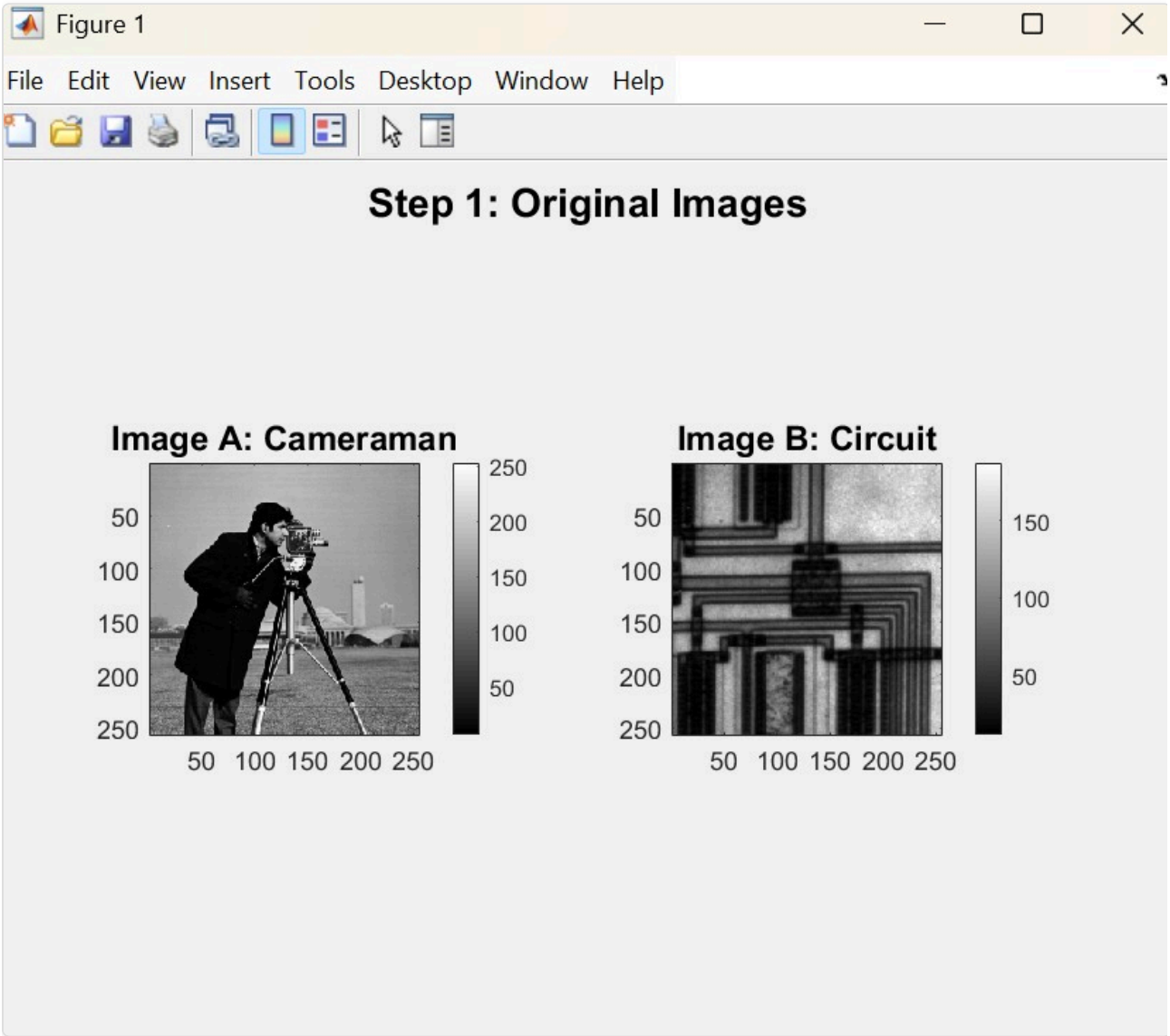

Figure 1: The two original images — Image A (Cameraman) and Image B (Circuit). These look completely different, making the swap experiment highly convincing.

4. Step 2: Compute the 2D Fourier Transform

Code

```
85  fftA = fft2(imageA);   % FFT of Image A: complex matrix of same size
86  fftB = fft2(imageB);   % FFT of Image B: complex matrix of same size
```

> **Lines 85–86:** `fft2()` computes the **2D Fast Fourier Transform**. It takes a real-valued 256×256 matrix (pixel intensities) and returns a **complex-valued** 256×256 matrix. Each element `fftA(u,v)` is a complex number of the form `a + bi`, where `a` is the real part and `b` is the imaginary part. Together, they encode one specific 2D sinusoidal pattern's amplitude and position in the image.
>
> **Why "Fast"?** The FFT algorithm computes the DFT in O(N log N) operations instead of O(N^2), making it practical for large images.

5. Step 3: Extract Magnitude and Phase

Code — Magnitude

```
94  magA = abs(fftA);      % Magnitude of Image A's spectrum
95  magB = abs(fftB);      % Magnitude of Image B's spectrum
```

> **Lines 94–95:** `abs()` computes the **magnitude** (absolute value) of each complex number: `|F(u,v)| = √(real² + imag²)`. The result is a real-valued matrix where each element tells you *how strong* that particular frequency component is. High magnitude = that frequency pattern contributes a lot of contrast/energy to the image.

Code — Phase

```
100  phaseA = angle(fftA);  % Phase of Image A's spectrum (radians)
101  phaseB = angle(fftB);  % Phase of Image B's spectrum (radians)
```

> **Lines 100–101:** `angle()` computes the **phase angle** of each complex number: `ϕ(u,v) = atan2(imag, real)`. The result is in **radians**, ranging from -π to +π. Each phase value tells you

where (at what spatial offset) that frequency pattern is positioned in the image. Phase encodes the **structural skeleton**: edges, textures, shapes, and the spatial arrangement of objects.

```
104 fprintf('=== Magnitude Statistics ===\n');
105 fprintf('Image A magnitude: min=%.2f, max=%.2f, mean=%.2f\n', ...
106     min(magA(:)), max(magA(:)), mean(magA(:)));
107 fprintf('Image B magnitude: min=%.2f, max=%.2f, mean=%.2f\n', ...
108     min(magB(:)), max(magB(:)), mean(magB(:)));
```

Lines 104–108: Print statistics about the magnitude spectra. `magA(:)` reshapes the 2D matrix into a single column vector so `min()`, `max()`, and `mean()` operate over *all* elements. `%.2f` formats the numbers to 2 decimal places. This helps verify that the FFT produced reasonable values.

```
110 fprintf('=== Phase Statistics ===\n');
111 fprintf('Image A phase: min=%.4f rad, max=%.4f rad\n', ...
112     min(phaseA(:)), max(phaseA(:)));
113 fprintf('Image B phase: min=%.4f rad, max=%.4f rad\n', ...
114     min(phaseB(:)), max(phaseB(:)));
```

Lines 110–114: Print phase statistics. The phase values should range approximately from `-3.1416` to `+3.1416` (i.e., `-π` to `+π`). `%.4f` gives 4 decimal places for higher precision.

6. Step 4: Reconstruct with Swapped Components

This is the core of the experiment. We recombine magnitude and phase in four ways:

> `X(u,v) = magnitude · exp(j · phase)`

Sanity Check 1: Magnitude(A) + Phase(A)

```
134  spectrum_magA_phaseA = magA .* exp(1i * phaseA);
135  recon_magA_phaseA    = real(ifft2(spectrum_magA_phaseA));
```

> **Line 134:** Reconstructs the complex spectrum from magnitude and phase using Euler's formula: `|X| · e^jϕ`. The `.*` operator performs element-wise multiplication. `1i` is MATLAB's imaginary unit. Since we use the *same* image's magnitude and phase, this should perfectly reconstruct the original.
>
> **Line 135:** `ifft2()` computes the **inverse 2D FFT**, converting back from frequency domain to spatial domain. `real()` discards the tiny imaginary residuals (on the order of 10^{-14}) caused by floating-point rounding — the true result is purely real.

Sanity Check 2: Magnitude(B) + Phase(B)

```
140  spectrum_magB_phaseB = magB .* exp(1i * phaseB);
141  recon_magB_phaseB    = real(ifft2(spectrum_magB_phaseB));
```

> **Lines 140–141:** Same as above but for Image B. Another sanity check — should reconstruct the circuit image perfectly.

Key Experiment 1: Magnitude(B) + Phase(A)

```
147  spectrum_magB_phaseA = magB .* exp(1i * phaseA);
148  recon_magB_phaseA    = real(ifft2(spectrum_magB_phaseA));
```

> **Lines 147–148: This is the critical experiment.** We take the *circuit's magnitude* but the *cameraman's phase* and reconstruct. **The question:** Will the result look like the circuit (magnitude source) or the cameraman (phase source)?
>
> **Answer:** It looks like the **cameraman** — proving phase dominates.

Key Experiment 2: Magnitude(A) + Phase(B)

```
154  spectrum_magA_phaseB = magA .* exp(1i * phaseB);
155  recon_magA_phaseB    = real(ifft2(spectrum_magA_phaseB));
```

> **Lines 154–155:** The reverse experiment. We take the *cameraman's magnitude* but the *circuit's phase*. **Result:** It looks like the **circuit** — again, phase wins.

7. Step 5: Display Original Images

```
161  figure(1);
162  sgtitle('Step 1: Original Images', 'FontSize', 14, 'FontWeight', 'bold');
164  subplot(1, 2, 1);
165  imagesc(imageA);
166  colormap gray;
167  axis image;
168  title('Image A: Cameraman', 'FontSize', 12);
169  colorbar;
```

> **Line 161:** `figure(1)` creates or selects Figure window #1.
> **Line 162:** `sgtitle()` adds a "super title" above all subplots.
> **Line 164:** `subplot(1, 2, 1)` divides the figure into a 1×2 grid and selects position 1 (left).
> **Line 165:** `imagesc()` displays a matrix as an image, automatically scaling the color range to fit the data.
> **Line 166:** `colormap gray` sets the colormap to grayscale (black=0, white=max).
> **Line 167:** `axis image` forces the aspect ratio to match the actual pixel dimensions (no stretching).
> **Line 168:** `title()` adds a title above this subplot.
> **Line 169:** `colorbar` adds a color scale bar showing the mapping from pixel value to displayed shade.

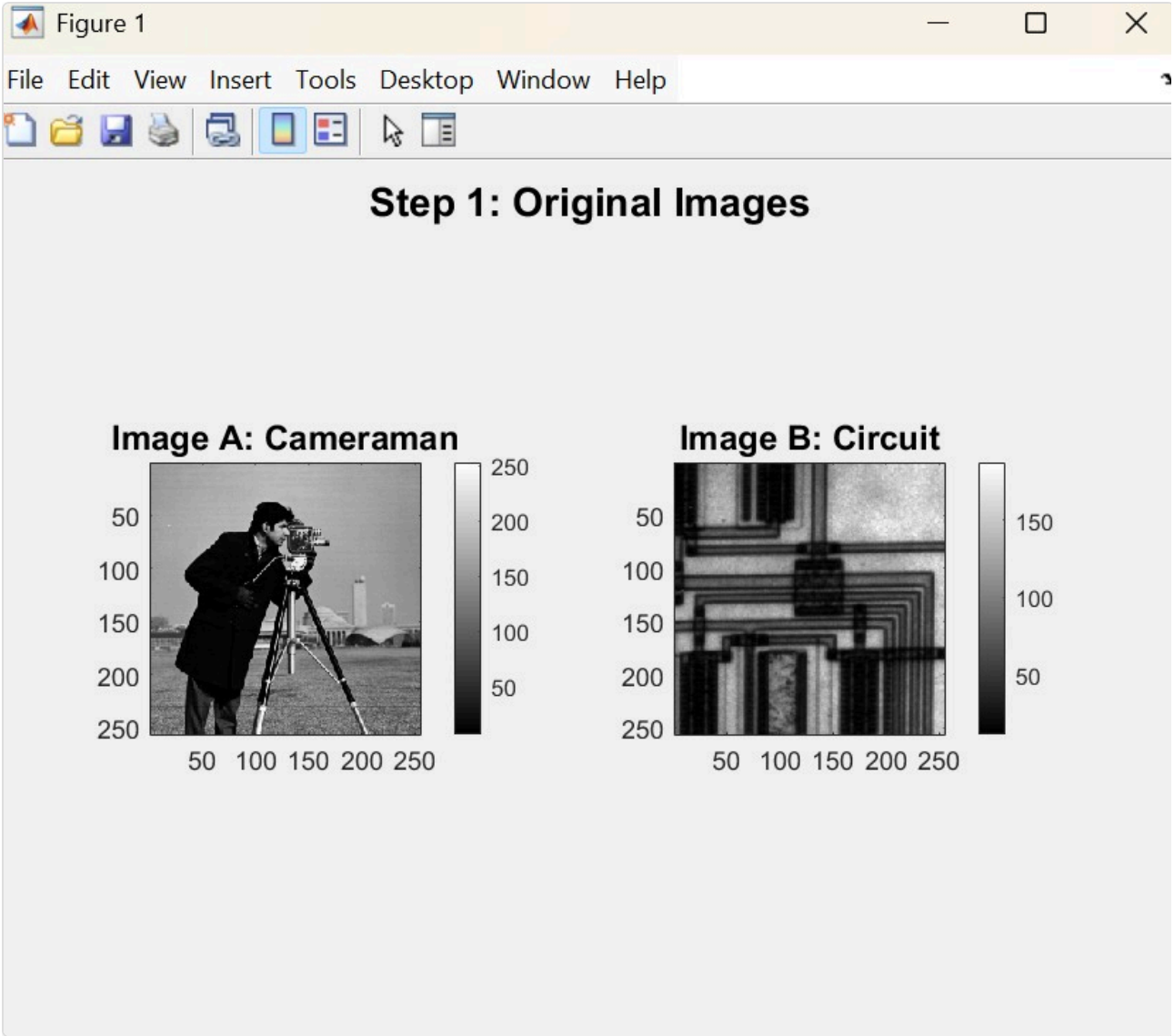

Figure 1: Original images. Left: Cameraman (256×256). Right: Circuit board (cropped to 256×256). These are standard MATLAB test images with very different visual content.

8. Step 6: Display Magnitude Spectra

```
189 imagesc(log(1 + fftshift(magA)));
190 colormap gray;
191 axis image;
192 title('|FFT(Image A)|', 'FontSize', 12);
193 colorbar;
```

> **Line 189 — two key functions:**
> • `fftshift()`: By default, MATLAB's FFT places the zero-frequency (DC) component at corner (1,1). `fftshift` rearranges the output so the DC component is at the **center** of the image, which is the conventional way to visualize frequency spectra.
> • `log(1 + ...)`: The DC component's magnitude is typically millions of times larger than high-frequency components. Taking the logarithm compresses this dynamic range so you can see detail across all frequencies. The `+1` prevents `log(0)` errors.

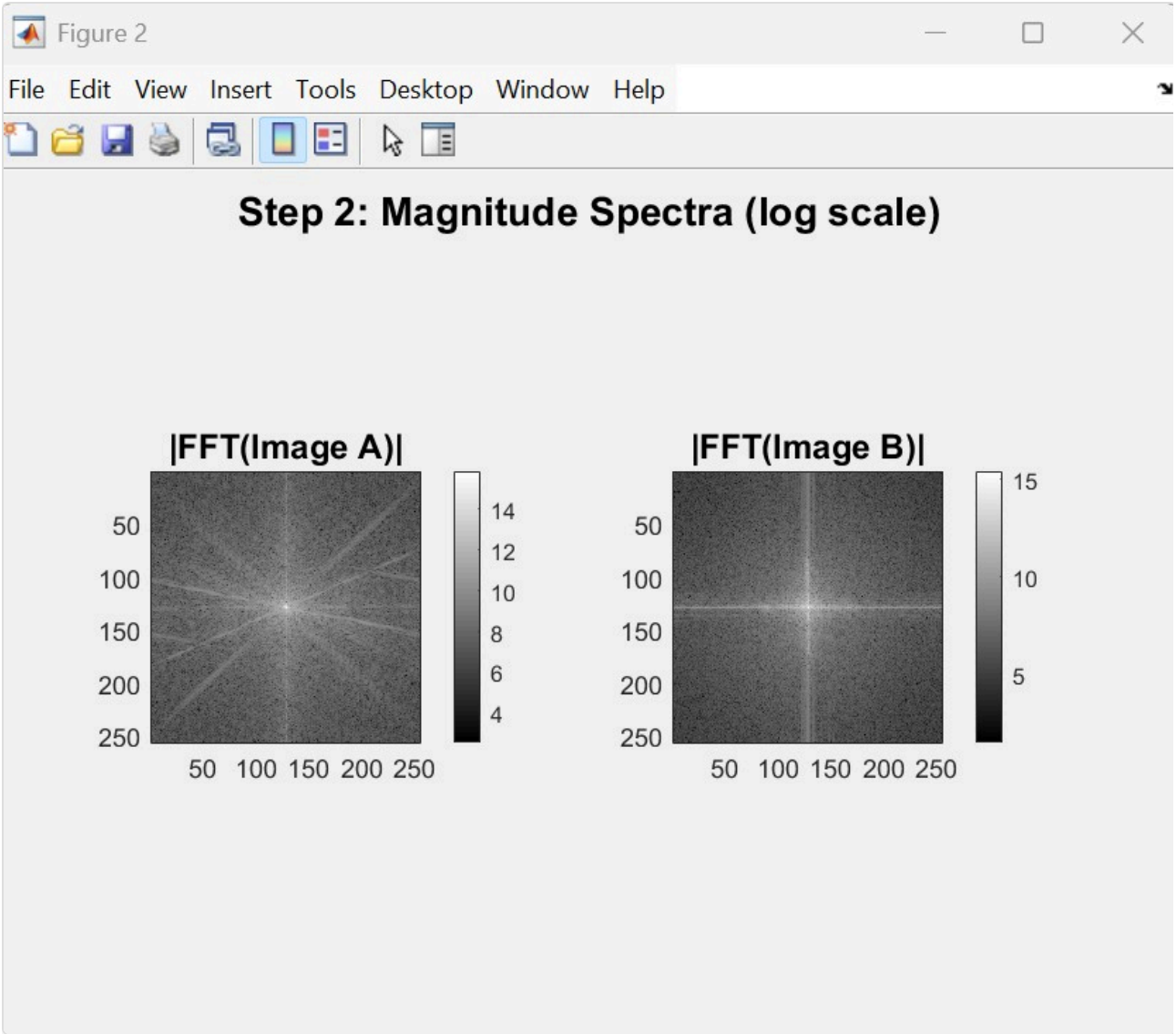

Figure 2: Magnitude spectra (log scale). The bright center is the DC component (average brightness). Lines radiating outward correspond to dominant edges. The circuit image shows strong horizontal/vertical lines matching its grid structure.

9. Step 7: Display Phase Spectra

```
212  imagesc(fftshift(phaseA));
213  colormap gray;
214  axis image;
215  title('Phase of FFT(Image A)', 'FontSize', 12);
216  colorbar;
```

> **Line 212:** We display the raw phase values (in radians, from -π to +π) after centering with `fftshift()`. No log scaling is needed because phase values already have a bounded range.

> **Important observation:** Phase spectra look like **random noise** to the human eye. This is deeply misleading — despite their noisy appearance, they encode *all* the structural information. This visual paradox is exactly what makes the experiment so powerful and surprising.

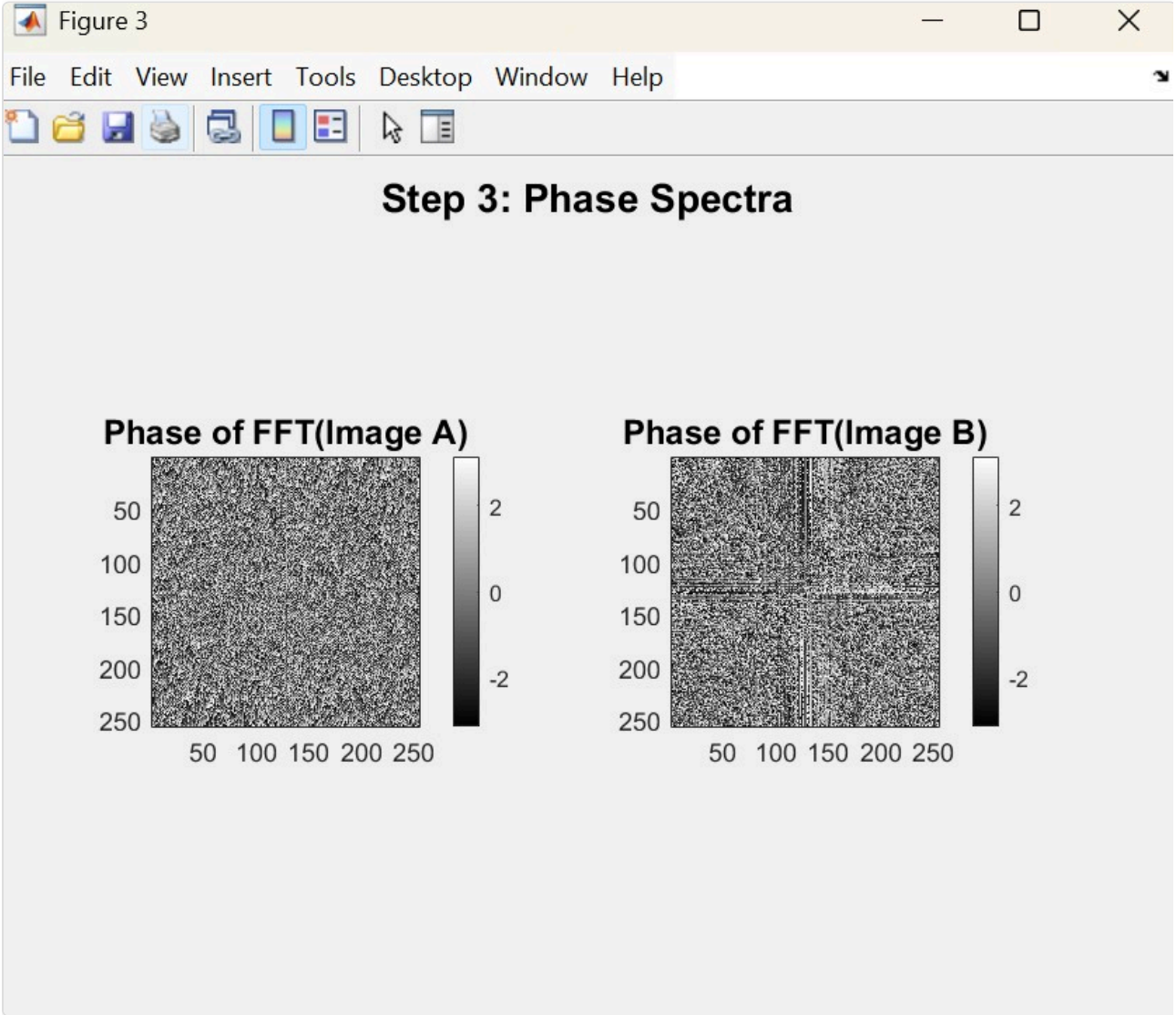

Figure 3: Phase spectra. They look like random noise, but contain all the structural information (edges, shapes, positions) needed to recognize the image. Look carefully at the right image — you can faintly see the circuit's grid pattern encoded in the phase.

10. Step 8: The Key Result — Visual Proof

```
237  subplot(2, 2, 3);
238  imagesc(recon_magB_phaseA);
239  colormap gray;
240  axis image;
255  title({'Mag(B) + Phase(A)', ...
256         'Looks like A! => PHASE WINS'}, ...
257         'FontSize', 11, 'Color', 'r');
```

> **Lines 237–257:** `subplot(2, 2, 3)` selects the bottom-left position of a 2×2 grid. We display the reconstruction that uses **Image B's magnitude** and **Image A's phase**. The title is colored **red** to draw attention to the key result. Despite having the circuit's energy distribution, the image clearly shows the cameraman — because phase controls the structure.

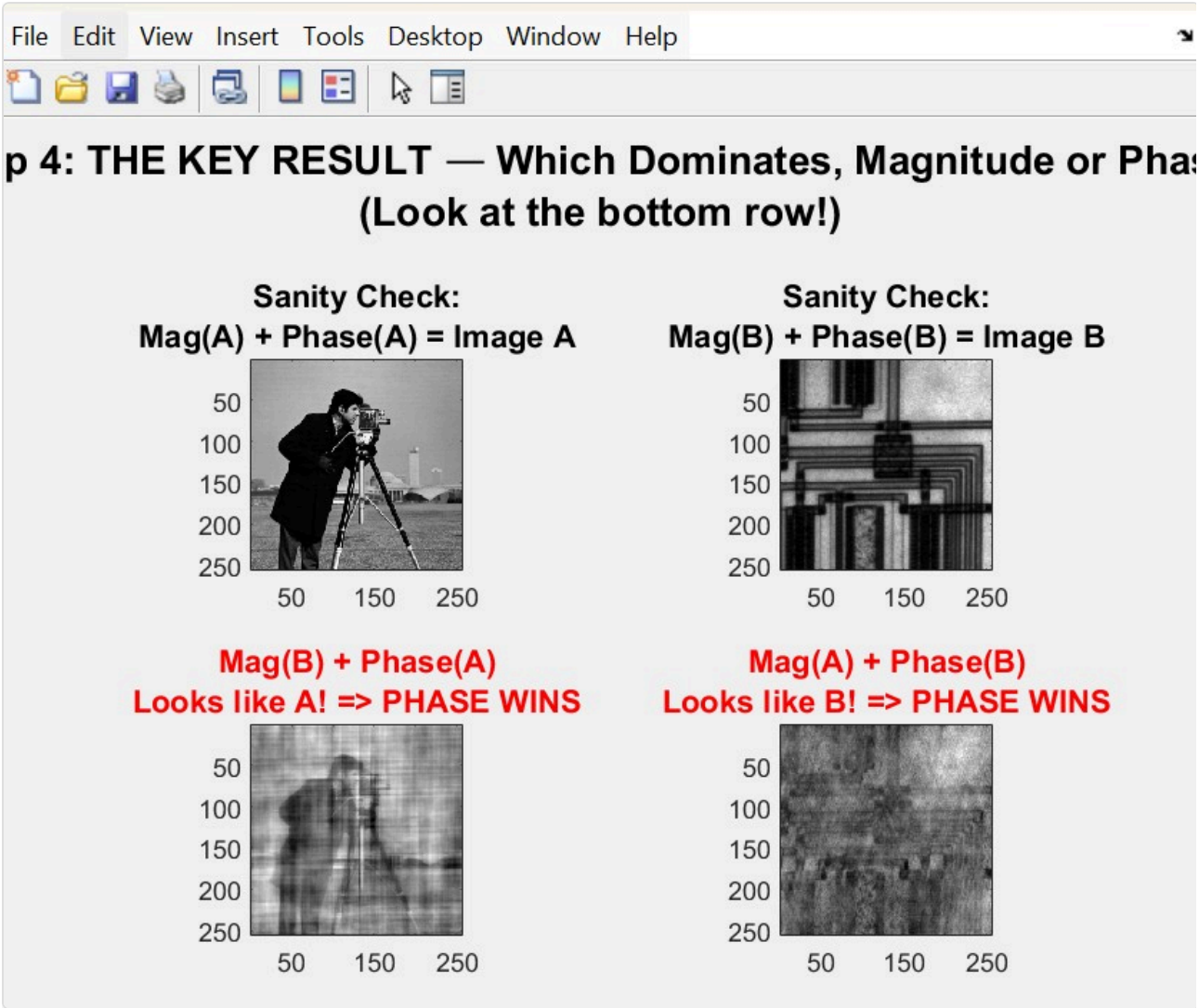

Figure 4: THE KEY RESULT. Top row: sanity checks (originals reconstructed perfectly). Bottom row: the proof. Bottom-left uses circuit's magnitude + cameraman's phase → you see the cameraman. Bottom-right uses cameraman's magnitude + circuit's phase → you see the circuit. ***PHASE WINS in both cases.***

Reconstruction	Magnitude From	Phase From	Result Looks Like
Top-Left (sanity check)	Cameraman	Cameraman	Cameraman ✓
Top-Right (sanity check)	Circuit	Circuit	Circuit ✓
Bottom-Left (experiment)	Circuit	**Cameraman**	**Cameraman! PHASE WINS**
Bottom-Right (experiment)	Cameraman	**Circuit**	**Circuit! PHASE WINS**

11. Step 9: Quantitative Verification

```
275  normA = imageA / max(imageA(:));
276  normB = imageB / max(imageB(:));
277  norm_magB_phaseA = recon_magB_phaseA / max(recon_magB_phaseA(:));
278  norm_magA_phaseB = recon_magA_phaseB / max(recon_magA_phaseB(:));
```

> **Lines 275–278:** Normalize all images to the range [0, 1] by dividing by the maximum pixel value. This ensures a **fair comparison** — without normalization, images with different brightness ranges would produce misleading correlation values. `max(imageA(:))` finds the single largest value across the entire 2D matrix.

```
281  corrWithA_case1 = corr2(norm_magB_phaseA, normA);
282  corrWithB_case1 = corr2(norm_magB_phaseA, normB);
```

> **Lines 281–282:** `corr2()` computes the **2D correlation coefficient** between two images. It returns a value between -1 and +1, where:
> • **+1** = perfectly identical
> • **0** = no correlation
> • **-1** = perfectly inverted
>
> We compare the "Mag(B) + Phase(A)" reconstruction against *both* originals. If the correlation with Image A (the phase source) is higher, it numerically confirms that phase dominates.

```
285  corrWithA_case2 = corr2(norm_magA_phaseB, normA);
286  corrWithB_case2 = corr2(norm_magA_phaseB, normB);
```

> **Lines 285–286:** Same comparison for the reverse swap: "Mag(A) + Phase(B)". The correlation with Image B (the phase source) should be higher.

Helper Function

```
314  function result = iif(condition, trueVal, falseVal)
315      if condition
316          result = trueVal;
317      else
318          result = falseVal;
```

```
319        end
320    end
```

> **Lines 314–320:** A simple helper function that acts as an inline if-else (ternary operator). MATLAB does not have a built-in ternary like C's `condition ? a : b`, so this function provides that functionality. It is used in the `fprintf` statements above to print "YES — Phase wins!" or "NO" based on the correlation comparison. The `function` keyword at the end of a script defines a **local function** (available since MATLAB R2016b).

12. Conclusion & Key Takeaways

Definitive Result

In **both** swap experiments, the reconstructed image resembles the image whose **PHASE** was used — not the image whose magnitude was used.

Therefore: Phase carries more perceptual information than magnitude.

Why Does Phase Dominate?

Aspect	Magnitude	Phase
What it controls	Contrast, brightness, energy	Edges, shapes, textures, positions
Human perception	Secondary (we adapt to contrast)	Primary (we recognize by structure)
If scrambled	Image looks different in contrast but is still recognizable	Image becomes completely unrecognizable
Information content	Lower — smooth, slowly varying	Higher — encodes spatial relationships between all frequencies

Analogy: Music

Think of the Fourier transform of music:

- **Magnitude** = the *volume* of each note
- **Phase** = the *timing/rhythm* of each note

If you play a song with the correct rhythm but wrong volumes, you still recognize the melody. If you play it with correct volumes but scrambled timing, it becomes unrecognizable noise. The same principle applies to images.

Real-World Importance

1. **Medical Imaging (MRI)**: MRI scanners measure magnitude and phase separately. Phase images are critical for flow quantification and susceptibility mapping.
2. **Image Compression (JPEG)**: JPEG preserves phase-related information (DCT coefficients' signs) carefully while aggressively quantizing magnitudes.
3. **Crystallography**: The "phase problem" — X-ray diffraction gives magnitudes but not phases — is the central challenge in determining molecular structures.
4. **Holography**: Holograms record *both* magnitude and phase of light waves, which is why they can reconstruct 3D scenes, unlike photographs that capture only magnitude.
5. **Adaptive Optics**: Telescopes correct for atmospheric distortion by measuring and compensating phase errors in the incoming light wavefront.

Summary of MATLAB Functions Used

Function	Purpose
`imread()`	Read image file into a pixel matrix
`double()`	Convert uint8 integers to double-precision floats
`fft2()`	Compute 2D Fast Fourier Transform
`ifft2()`	Compute inverse 2D FFT (frequency → spatial domain)
`abs()`	Compute magnitude of complex numbers
`angle()`	Compute phase angle of complex numbers (radians)
`exp(1i * ...)`	Euler's formula: convert phase back to complex exponential
`fftshift()`	Center the zero-frequency component for visualization
`imagesc()`	Display matrix as image with auto-scaled colors
`subplot()`	Create multi-panel figure layouts
`corr2()`	Compute 2D correlation coefficient between two matrices
`real()`	Extract real part (discard floating-point rounding artifacts)
`log()`	Natural logarithm (used for dynamic range compression in display)

13. References

1. Oppenheim, A. V., & Lim, J. S. (1981). “The Importance of Phase in Signals.” *Proceedings of the IEEE*, 69(5), 529–541.
2. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing*, 4th Edition. Pearson.
3. Haykin, S., & Van Veen, B. (2007). *Signals and Systems*, 2nd Edition. Wiley.